

BotCortex: Sim-to-Real and Cross-Embodiment Research

OpenHorizon Labs · Research dossier · 12 September 2026

Selected direction: transfer diagnosis on OpenArm and the Waveshare RoArm-M2-Pro.

Working paper title: *Repair the Program or Repair the Twin? Active Failure Diagnosis for Cross-Embodiment Skill Transfer.*

Status: literature-informed research proposal and implementation plan. Existing BotCortex simulation results are identified separately from proposed experiments. No cross-robot hardware result or novel-method result is claimed.

1. Executive decision

Build a system that determines **which part of a failed robot deployment needs repair**, then measures whether that decision reduces the number of real-world attempts needed to recover. Use BotCortex's agent-authored programs, local execution, and failure records as the experimental platform.

The first paper should answer a narrow question:

When a task fails after transfer to another robot or from simulation to hardware, can primitive-level contracts, counterfactual replay, and targeted diagnostic measurements select a better intervention than repeatedly rewriting the program or repeatedly recalibrating the simulator?

The selected hardware scope is **OpenArm plus the Waveshare RoArm-M2-Pro** as the second physical arm, with an **Intel RealSense depth camera** (already owned, to be mounted) as the shared object-pose sensor. The RoArm-M2-Pro is a low-cost 4-DoF desktop arm with serial-bus servos and a JSON command interface over serial or Wi-Fi, which gives a deliberately large embodiment gap from the 7-DoF, CAN-driven bimanual OpenArm. Reach, payload, control rate, and repeatability must be measured during bring-up and recorded in its capability pack rather than copied from marketing pages. A third simulated embodiment can broaden controlled experiments, but does not replace the second physical system.

The proposed intervention set is: repair the program/planner; repair perception or frame calibration; identify controller/physical parameters; learn a bounded residual; or abstain and ask for assistance. The contribution must be the **selection procedure and its measured recovery efficiency**, rather than the existence of these familiar repair tools.

Why this direction

- It addresses an immediate product problem: a failed rehearsal or deployment should produce an evidence-based next step, not another unsupported motion guess.
- It fits the architecture: the agent authors and repairs; local executors run the task; customer-owned memory records what happened.
- It admits a bounded experiment with two arms and a few task families, without requiring a new foundation model or a large dexterous pretraining program.
- It has strong nearby research. Fail2Progress already uses simulated failure reconstruction; COMPASS already diagnoses simulator parameters; ASID already chooses identification actions. These are starting points and novelty constraints, not missing literature. [3](#), [6](#), [8](#), [10](#)

Deliverables and decision gates

Deliverable	Evidence required
Portable execution substrate	Two adapters pass the same conformance suite, with explicit capability differences.
Transfer-failure benchmark	Versioned tasks, held-out faults, independent outcome verification, and controlled labels.
Active repair selector	Fewer hardware trials or human minutes than simple strategies under matched budgets.
Reproducible paper artifact	Programs, configurations, traces, split definitions, analysis, and representative failures.

If a fixed strategy or a simple rule-based selector performs equally well, report that result and revise the method. A useful SDK can still ship without a strong algorithmic novelty claim.

2. Evidence scope and current BotCortex baseline

Research scope

The discovery pass screened **85 distinct URLs, including 31 arXiv links**. It combined search, primary paper pages, methods/results sections and selected appendices, repository inspection, and official conference pages. This is a focused review, not an exhaustive survey or a claim that every discovered page was read cover-to-cover.

The bibliography distinguishes core technical sources from background discovery. Paper versions are pinned where recorded. A source's reported experiment is not a reproduced BotCortex result. Search coverage alone cannot establish novelty; citation tracing and an updated review are required before submission.

Implemented and previously verified

BotCortex runtime **0.0.2**, source commit `1f921978fab2c34962dba4daf55cc7b7e4d00eef`, fixes a concrete manipulation failure in native MuJoCo and browser WASM. The captured request was “Move the blue block next to the red one.” Two underlying defects mattered:

1. Geometry queries could restore an asymmetric two-finger grasp incorrectly, changing physical state merely by asking an IK or tip-position question.
2. Transit validation could accept accurate endpoint arrival while sweeping another block away.

The fix makes geometry queries preserve full simulation state and adds a deterministic object-motion guard. Candidate routes reject lost grasps and unintended displacement of unheld objects above 15 mm; alternate routes are tried before refusal. This is documented in `docs/SIMULATION_FAILURE_FIX.md`.

Evidence	Recorded result and scope
Captured simulated transfer	Blue moved from <code>[0.32, -0.05, 0.222]</code> m to <code>[0.2248, -0.2246, 0.22]</code> m.
Bystander preservation	Red and green retained horizontal position; 2 mm vertical movement was initial settling.
Backend coverage	Native MuJoCo and the shipped browser WASM wheel passed the reproduced task and tested variants.
Earlier verification	228 runtime tests; 164 web tests; typecheck/build; 20 fixture smoke checks; physical simulation smoke.
Observed latency	About 59–70 seconds initial computation, followed by roughly 30 seconds paced playback. These are separate quantities.
Hardware transfer	Not established by these tests. No new hardware evaluation is reported here.

These checks were run during the implementation work, not rerun to generate this document. The current result is a useful regression case, not a benchmark-wide success rate.

Portability gaps found in the source

Only the OpenArm platform is implemented. Descriptors do not yet make every backend generic: joint naming/count, gripper assumptions, IK, and backend mappings still contain OpenArm-specific behavior. Platform configuration and limits need cleaner instance ownership. Scene queries use privileged simulator state. A functioning `RealRobot` execution path is absent, and CLI `--execute` is explicitly unwired in the inspected implementation.

The research therefore needs real adapter and sensing work before hardware experiments. It should not present the existing browser simulator as an already-portable deployment SDK.

3. The two anchor references

Play2Perfect: what transfers, and what it costs

Play2Perfect: What Matters in Dexterous Play Pretraining for Precise Assembly? was reviewed at arXiv [2606.26428v3](#), dated 8 September 2026. It learns a dexterous prior from task-agnostic simulated play and finetunes it for precise assembly. [1](#)

The real platform is a **7-DoF KUKA arm with a 22-DoF Sharpa hand**. Reported operating rates are **60 Hz policy inference**, **30 Hz object tracking**, and **120 Hz training physics**. Play pretraining takes approximately **seven days on one RTX A6000**, using **24,576 parallel environments**; downstream finetuning takes about **one day per task**, using **12,228 environments**. Faster downstream learning is meaningful, but the reusable pretraining cost must remain visible.

Reported hardware results include:

Experiment	Author-reported outcome	Interpretation
Tight insertion, 10 mm clearance	10/10	Small trial count on the tested setup.
Tight insertion, 2 mm clearance	9/10	Tighter contact constraints reduce reliability.
Tight insertion, 0.5 mm clearance	6/10	Precise assembly remains difficult despite transfer.
Beam assembly, separately evaluated steps	8/10 and 7/10	Per-step rates are not end-to-end completion.
Additional end-to-end beam assembly	12/20	Separate appendix evaluation; do not replace it with step rates.
Screw-leg insertion / full screwing	7/10 / 5/10	Insertion and complete task success differ.

The policies use object-pose estimates and externally specified task sequencing and goal poses. Occlusion and contact mismatch remain failure sources. The paper describes short-horizon assembly skills, not a complete autonomous general-purpose assembly system.

The MIT-licensed repository provides an inference/deployment core, while actual deployment still requires matching observation layouts, action scaling, object tracking, robot I/O, and controllers. Its deployment guide is not a turnkey hardware stack. The project also has an interactive browser MuJoCo demonstration; browser simulation itself is not a research contribution.

BotCortex takeaway: a pretrained dexterous policy can eventually be one local executor. Do not couple the first diagnosis paper to reproducing this expensive pretraining regime or to acquiring an equivalent dexterous hand. Reuse the ideas of structured observations, contact-sensitive evaluation, and explicit transfer assumptions.

DexGPT: reconstruction is not execution success

The reviewed repository is `Hu-xiao-max/dexgpt`, commit `03ba8a26eaf9dfd272ac1a22c0fd06ccb9b2e5b`, described as “GPT for real2sim.” It reconstructs and retargets a monocular human GIF to two Sharpa hands in MuJoCo. The review inspected the README, retargeting and rollout code, and the stored physics report. [2](#)

Its saved report contains `task_success: false`, maximum penetration **5.623 mm** against a **less-than-5 mm** criterion, peak simulated contact force **314.7 N**, and hinge RMSE **5.177 degrees**. These numbers are from that saved simulation report, not new measurements or evidence of safe hardware forces.

The reconstruction assumes scale/depth, fixes the object's base, and prescribes wrist motion in place of a complete arm controller. No hardware execution controller was found in the reviewed path. A visually plausible motion or kinematic replay therefore cannot establish manipulation success, feasible arm dynamics, or real-world transfer.

No top-level license was identified in the reviewed snapshot; repository metadata returned no license. Third-party asset licenses are separate. Treat reuse permission as unresolved before incorporating its code or assets.

BotCortex takeaway: demonstrations can initialize object relations, task hypotheses, and candidate motions. They must then pass explicit kinematic, contact, effect, and hardware checks. Preserve failed physics reports rather than promoting a rendered video to a successful trial.

4. Related work and the novelty boundary

The following matrix focuses on overlaps that can invalidate an overly broad claim. It is a positioning aid, not a ranking of papers or a substitute for reproducing their methods.

Source	Relevant prior contribution	Boundary for BotCortex
ASPIRE 3	Trace-guided program repair, reusable skills, evolutionary search, cross-embodiment examples.	Code repair and a growing skill library are already established.
CaP-X 4	Coding-agent evaluation with perception/control services and real deployment.	Generating robot code through an agent harness is not enough.
Nautilus 5	Typed policy/robot/environment contracts, isolated execution, reproducible interfaces.	Interface portability is not equivalent to unchanged-policy task success.
Fail2Progress 6	Reconstructs failures and generates targeted simulated data to improve skill-effect models.	Replay-based failure classification already has direct precedent.
Counterexample-guided abstraction repair 7	Repairs symbolic/geometric abstractions from counterexamples.	Contract violations driving repair are not intrinsically new.

Source	Relevant prior contribution	Boundary for BotCortex
COMPASS / What Went Wrong? 8	Causal identification of simulator parameters responsible for real/sim trajectory mismatch.	Physics-parameter diagnosis must be compared with existing causal calibration.
RAPT 9	Predictive out-of-distribution detection, failure localization, semantic diagnosis.	Monitoring plus a natural-language explanation is insufficient novelty.
ASID 10	Active exploration for system identification.	Choosing informative calibration actions is prior art.
Phys2Real 11	Physical priors and interactive online adaptation with uncertainty.	Uncertainty-aware physical adaptation is not an untouched area.
Neural Fidelity Calibration 12	Informative online simulator adaptation.	A better twin alone needs a specific advance over adaptation methods.
TRANSIC 13	Sim-trained base policy plus a gated residual learned from human corrections.	Residual recovery and human-in-the-loop transfer already exist.
HIL-SERL 14	Real-world learning from demonstrations, interventions, and learned rewards.	Direct hardware teaching is a viable lane, not a new contribution by itself.
RialTo 15	Real-to-sim-to-real robustification using reconstructed environments.	Building and training in a digital twin has close precedent.
CEI 16	Functional 3D representations and aligned trajectories across embodiments.	Transferable representations must be distinguished from general adapter design.
UHAS 17	A shared hand action space for cross-embodiment manipulation.	Shared actions still need embodiment-specific dynamics and controller calibration.
ReKep 18	Relational keypoint constraints and closed-loop optimization.	Object-centric constraints and geometric replanning are established ideas.

The closest scientific collision

Fail2Progress is particularly important: simulated reconstruction can distinguish inaccurate symbolic predictions from a sim-to-real gap, and the work generates failure-targeted data for effect-model improvement. The investigated formulation leaves correction of the sim-to-real-gap category open. BotCortex cannot claim to invent “replay a failure to decide what went wrong.” [6](#)

The narrower opportunity is to evaluate a **joint choice among program, perception/calibration, physical adaptation, and residual-policy repairs**, with active observations when the initial evidence is ambiguous. The selector must outperform equal-budget alternatives on both known and held-out failures across physical embodiments.

COMPASS and ASID constrain the other side of the novelty claim: identifying causal simulator parameters and choosing informative identification motions are existing methods. The proposed contribution sits at the boundary between these physical repairs and program-level repairs, rather than claiming either family from scratch. [8](#), [10](#)

Additional context

Grounding Sim-to-Real Generalization, DrEureka, Re³Sim, and RoboInspector inform transfer evaluation, randomization, reconstructed environments, and failure feedback. They are included as supporting references, with the reading scope stated in the bibliography. No headline metric from these sources is treated as a BotCortex result. [19–22](#)

5. What “portable” and “unchanged” must mean

A task can retain its meaning across robots while requiring different joint trajectories, grasp choices, controllers, or policy weights. A robot with a different reach, gripper, force capability, or sensing setup may not be capable of the same task at all.

Transfer claim	What stays fixed	What may legitimately change
Task-semantic transfer	Goal, allowed effects, invariants, parameter meaning.	Robot-specific motion and executor selection.
Program transfer	Exact task source hash.	Adapter, calibration, planner output, declared bindings.
Policy transfer	Exact weights and observation/action contract.	Only explicitly reported preprocessing and controller bindings.
Adapter compatibility	API and conformance behavior.	Embodiment-specific driver and kinematics.
Trajectory replay	The stated trajectory representation and timing.	Retargeting changes must be disclosed; joint replay is generally robot-specific.

Record separate hashes for **task source**, **policy weights**, **adapter**, **calibration**, and **simulator/model**. “Zero-shot” must name the quantity that received no target-domain optimization and disclose all target-specific setup. Calibration-free, demonstration-free, unchanged weights, and unchanged program are different claims.

Example: blue beside red

The task intent should express blue being supported by the table, in a specified spatial relation to red, with uncertainty-aware tolerances. During carrying, require maintained grasp. Preserve objects declared as bystanders. Verify the final state after release and a settling interval.

The existing no-unheld-object-motion transit guard is appropriate for this pick-and-place case. It is not a universal manipulation contract: **intentional pushing must allow motion of the designated pushed object**. A task-specific allowed-effect set prevents a useful guard from banning the task itself.

Feasibility is a first-class result

Binding a skill to a platform should produce one of: supported; supported after specified calibration or executor adaptation; unsupported; or insufficient evidence. Do not silently replace a missing force controller with a position approximation and call the program portable.

The product promise is reusable task intent and controlled adaptation on supported robots. It is not guaranteed identical behavior on arbitrary hardware.

6. Research questions and falsifiable hypotheses

Primary hypothesis

Given matched repair operators, observations, and attempt budgets, a selector using contracts, counterfactual replay, and targeted measurements reduces **real-world attempts to verified recovery** compared with fixed repair strategies and trace-only selection.

Secondary questions

1. Do object-centric contracts improve transfer diagnosis beyond raw state/action traces?
2. Does active measurement add value beyond passive replay, after charging its real-world cost?
3. Can diagnosis transfer across robots when controller, calibration, and model versions are explicit?
4. Can uncertainty-aware abstention lower wrong-repair and false-success rates without making the system unhelpfully reluctant?
5. When is a program edit preferable to simulator calibration or a residual policy, and when is the cause not identifiable from available sensing?

Candidate failure classes

Class	Examples	Candidate intervention
Program / planner	Wrong frame usage in code, infeasible waypoint sequence, missing prerequisite, incorrect effect assumptions.	Edit the failing step, regenerate the plan, or repair the effect model.
Perception	Wrong object identity, occlusion, stale pose, biased tracking.	Reobserve, change viewpoint, repair perception configuration.
Frame / TCP calibration	Camera-to-base or tool-center offset, unit convention mismatch.	Validate transforms and recalibrate the affected mapping.

Class	Examples	Candidate intervention
Controller / physics	Tracking delay, gain mismatch, friction, mass, compliance, grasp/contact discrepancy.	Identify parameters, update the twin/controller binding, or use a bounded residual.
Compound / unresolved	Multiple causes or observationally equivalent explanations.	Select an informative measurement or abstain.

Infrastructure failures such as stale workers, dropped commands, or mismatched artifacts must not be mislabelled as physical dynamics. Detect them before entering the physical diagnosis model, and retain their traces for operational analysis.

Identifiability limits

A misplaced object can be explained by a bad camera transform, a wrong TCP, tracking error, a bad pose estimate, or flawed program coordinates. Replaying one trajectory in one simulator does not establish which explanation is causal. The method needs competing hypotheses and measurements that can discriminate between them.

The research claim should therefore be **effective intervention selection under measured uncertainty**, not universal causal identification. Controlled injected faults permit stronger label-based evaluation; naturally occurring failures need independent adjudication and may remain unresolved.

7. Proposed method: diagnose, measure, repair

This section specifies a design to implement and test. It is not a description of an already-trained selector.

Step 1 — Capture a replayable evidence packet

At every primitive boundary, record task and artifact versions, intended action, measured action/state, timestamps, object estimates and uncertainty, controller status, contacts when observable, contract verdicts, human overrides, and outcome evidence. Keep raw observations or referenced recordings sufficient to revisit disputed diagnoses.

The executor and verifier must agree on run identity and timebase. Missing observations are explicit missing data, not zero values. The verifier produces its result independently of the authoring agent's explanation.

Step 2 — Localize the first supported violation

Evaluate preconditions, invariants, and postconditions with measurement validity and tolerances. Examples include grasp maintained during transit, object motion consistent with the allowed-effect set, and the target supported at the intended location after release.

Distinguish “predicate false” from “cannot observe predicate.” Joint arrival alone does not prove grasp or placement. The first visible violation is a useful localization point, but is not automatically the root cause.

Step 3 — Construct competing counterfactuals

Replay the relevant primitive or short segment from the reconstructed state, varying one candidate explanation at a time where possible. Examples: correct the waypoint while holding calibration fixed; change the TCP estimate; vary tracking latency; or perturb friction within identified bounds.

Use ensembles when state reconstruction or physics parameters are uncertain. A counterfactual that explains the trace is a candidate explanation, not proof. Geometry and diagnostic queries must preserve simulator state; the 0.0.2 regression demonstrates why this requirement matters.

Step 4 — Choose a targeted measurement

When hypotheses remain ambiguous, select a bounded diagnostic observation or action: reobserve a fiducial or object; check a known tool pose; execute a low-amplitude tracking probe; or test contact at an operator-approved configuration.

A starting decision rule can rank candidate measurements by expected hypothesis reduction relative to time and execution cost. Compare it with random probes and a simple rule-based choice. Do not label a hand-written heuristic a learned causal policy.

Step 5 — Select the repair

Estimate each available repair's chance of recovery, hardware-attempt cost, human time, and applicability. A conceptual objective is:

```
choose repair minimizing:
  expected attempts to verified recovery
  + weighted human time
  + weighted execution cost
subject to capability and operating constraints
```

The weights and treatment of failed or censored episodes must be fixed before final evaluation. Operators unavailable on a robot are masked, not imputed as successful. An abstention is an explicit decision, counted in coverage and cost.

Step 6 — Re-execute and verify

Apply the chosen intervention as a versioned change. Rehearse when supported, then run a supervised hardware attempt within the experiment protocol. Check task effects independently. A fluent explanation, completed function call, or simulation pass is insufficient to label hardware success.

Step 7 — Store the intervention outcome

Memory must include the pre-repair evidence, hypothesis, chosen intervention, changed artifacts, and verified result, including failed repairs. Retrieval conditions on robot capabilities and relevant versions. Separate training, validation, and test memory; freeze or explicitly constrain online updates to prevent leakage.

8. Portable SDK and runtime architecture

Architectural rule

The LLM is outside the real-time control loop. It authors or repairs a task, while local primitives, planners, or learned policy executors run it. Teaching normally uses frontier cloud models; taught execution and local memory should not depend on a live model call. A local authoring model remains an optional configuration.

This preserves the BotCortex product direction: chat authors tasks, a plan view supports review, and the runtime owns execution. The hosted web UI should not be described as an already-offline interface merely because execution can be local.

Capability packs

A robot descriptor is necessary but insufficient. A capability pack should contain:

- Kinematic tree, collision geometry, joint ordering, units, tool frames, TCP, and gripper semantics.
- Position/velocity/torque limits, available control modes, measured control rate and latency, and stop behavior.
- Sensor types, calibration, timestamp sources, observation conventions, and uncertainty estimates.
- Identified dynamics and contact assumptions where relevant, with confidence and validity range.
- Driver, firmware, model, controller, calibration, and schema versions.
- Supported primitives, policy interfaces, task affordances, and conformance-test results.

Two robots can expose the same primitive name while providing different capabilities. Bind only when the required semantics are supported; do not hide absent sensing or unsupported control modes behind a uniform function signature.

Component boundaries

Component	Responsibility	Required output
Platform adapter	Translate bounded runtime commands into robot-specific I/O.	Measured state, acknowledgements, timing, status.
Perception / frame service	Maintain object estimates and transform validity.	Timestamped estimates with uncertainty and provenance.

Component	Responsibility	Required output
Skill contract layer	Declare goals, allowed effects, invariants, and capability requirements.	Machine-checkable contract and binding result.
Executor registry	Select primitive, planner, task policy, or VLA executor.	Versioned executor and declared observation/action contract.
Simulator service	Rehearse and replay controlled hypotheses.	State snapshots, traces, contract outcomes, model identity.
Evidence recorder / verifier	Record execution and independently determine outcomes.	Auditable episode and verdict.
Diagnosis / repair service	Compare hypotheses, request measurements, select intervention.	Decision, uncertainty, rationale grounded in evidence.
Local memory	Retain lessons and intervention outcomes.	Version-aware retrieval with split isolation.

Implementation direction

Start with OpenArm, then implement a second adapter against the same conformance suite. Remove joint-count, naming, and gripper assumptions from shared planning and simulation logic. Make platform state instance-specific so two different robots can coexist in a process or evaluation harness.

Use native/headless simulation for repeated training and diagnosis experiments. Keep browser simulation for inspection, onboarding, and interactive rehearsal. Browser and native paths should share semantics and contract fixtures, but performance equality is not assumed.

Keep learned policies swappable. BotCortex need not train a foundation VLA; use policies as bounded executors underneath the task program. Pin weights and preprocessing separately from task code and adapter configuration.

9. Two teaching and adaptation lanes

Lane A — Simulation-first transfer

1. Bring up the robot adapter and calibrate the observed scene and frames.
2. Author object-centric intent and bind it to supported executors.
3. Rehearse in a versioned native simulator, including uncertainty in relevant parameters.
4. Review the candidate and perform supervised hardware trials.
5. Diagnose discrepancies, choose an intervention, and update the relevant artifact.
6. Save the verified skill and its operating envelope locally.

Play2Perfect, RialTo, TRANSIC, and calibration methods supply useful patterns, but have different prerequisites and costs. Do not compare their learning efficiency without accounting for pretraining, demonstrations, environment reconstruction, and human corrections. [1](#), [10–15](#)

Lane B — Direct hardware teaching

1. Collect demonstrations through a supported teleoperation or kinesthetic-teaching interface.
2. Record observations and **actually executed actions**, including human overrides and timestamps.
3. Train or adapt a bounded task executor, with a validated reward/outcome mechanism.
4. Use supervised interventions to gather recovery data.
5. Evaluate the executor on held-out initial states and store its capability envelope.

HIL-SERL and TRANSIC are relevant precedents for human-in-the-loop learning and residual correction. Their reported success should not be generalized to arbitrary robots, unseen tasks, or uninstrumented environments. [13](#), [14](#)

How the lanes meet

Both lanes use the same evidence schema, task contracts, artifact versioning, and outcome verifier. A simulation-first policy may acquire a residual from hardware corrections; a directly taught skill may later use simulation for diagnosis. Report the actual training path rather than assigning a misleading single label.

For the first paper, use a small, fixed repair-operator set. A bounded residual operator is optional if it would otherwise dominate schedule and confound the selector study. If omitted, narrow the final hypothesis and baseline matrix accordingly rather than claiming to evaluate it.

10. Experimental design

Robots and tasks

Use OpenArm and the Waveshare RoArm-M2-Pro. Both cover tabletop manipulation, and the embodiment difference is large: joint count (7 vs 4), actuation and bus (CAN motors vs serial-bus servos), gripper geometry, reach, and control rate all differ. The RoArm-M2-Pro's smaller workspace and lower stiffness bound the shared task set; a task the small arm cannot reach or hold is recorded as unsupported for that platform, not forced. The Intel RealSense depth camera provides object-pose observations for both arms; mount and calibrate it once per workcell and record the camera-to-base transform for each arm separately. Match task semantics and record differences in reach, gripper geometry, control rate, and observability.

Start with four candidate task families: relational pick-and-place; pick/transport around bystanders; deliberate object pushing; and a contact-sensitive placement or insertion task supported by both setups. Final selection follows the hardware pilot. Do not force a task onto a robot missing its required sensing or mechanics.

Failure benchmark

Fault group	Controlled examples	Important separation
Program / planning	Incorrect waypoint, missing prerequisite, wrong object relation.	Distinguish actual implementation errors from unreachable tasks.
Perception	Pose bias, occlusion, delayed observation, identity confusion.	Preserve measurement-validity labels.
Frame / TCP	Known transform or tool offset.	Keep injected configuration hidden from the selector.
Tracking / latency	Bounded delay, gain or velocity mismatch.	Verify actual response instead of assuming an injected setting took effect.
Contact / grasp	Object mass, surface friction, grasp offset, compliance changes.	Stay within the hardware operating envelope.
Compound / held-out	Unseen combinations and naturally occurring failures.	Allow ambiguous or multiple causes.

Validate controlled faults in simulation first. Hardware injections must be bounded, supervised, and appropriate to the mechanism. Unsafe or physically destructive faults are unnecessary to test the hypothesis.

Baselines under shared resources

1. Always repair the program/planner.
2. Always recalibrate or identify physical parameters.
3. Always use residual adaptation, **if that repair operator is implemented for all compared selectors.**
4. A Fail2Progress-inspired classification/repair strategy, with adaptations clearly stated.
5. A trace-only selector with no counterfactual replay or active measurements.
6. A simple rule-based selector over the same evidence and operators.
7. The proposed active selector.
8. An oracle-class selector for controlled faults, treated as a reference rather than a deployable method.

Use the same candidate repair implementations and comparable observations, prompts/models, attempt limits, compute budgets, and available demonstrations wherever the comparison permits. Charge diagnostic actions against the relevant hardware/time budget. A published-method reproduction and a locally adapted baseline must be labelled differently.

Required ablations

- Remove contracts; retain traces.
- Remove counterfactual replay.
- Replace active measurement with passive evidence or random probes.
- Remove uncertainty-aware abstention.

- Remove cross-robot memory transfer.
- Remove version conditioning from retrieval.

Use a staged design rather than running every ablation on every possible hardware condition immediately. Simulation can screen hypotheses; final claims need the relevant physical comparisons.

Splits and execution protocol

Freeze task/robot/fault combinations into training, validation, and test partitions. Include held-out combinations, not merely fresh random seeds of the same scenarios. Randomize or counterbalance strategy order and record object resets, calibration changes, operator identity, and session/day effects.

For a concrete planning example, **2 arms × 4 tasks × 30 trials × 4 strategies = 960 evaluation episodes**. This is an illustrative core comparison, not a finalized power calculation or the entire baseline/ablation suite. Recovery attempts and diagnostic probes can multiply the actual physical workload.

11. Metrics, statistics, and outcome verification

Primary and secondary measurements

Metric	Operational definition
Trials to verified recovery	Hardware task attempts from initial failure to independently verified success; report budget exhaustion.
Fixed-budget success	Fraction recovering within a prespecified attempt/time budget.
Human minutes	Setup, diagnosis, intervention, reset, and correction time, with categories visible.
Wrong-repair rate	Chosen intervention inconsistent with controlled fault labels or independent adjudication.
False-success rate	System reports success when the independent verifier or review finds failure.
Abstention and coverage	Frequency of refusing a diagnosis/execution and fraction of eligible cases actually handled.
Unchanged-program rate	Fraction transferred without modifying the task source hash; report adapter/calibration/weights separately.
Effort and runtime	Calibration, demonstrations, training compute, authoring, planning, simulation, hardware execution, and playback.

A repair can be practically effective without proving the original causal explanation. Report diagnosis accuracy and recovery performance separately. Also report failures to recover; do not calculate average recovery time only over successful episodes without clearly identifying that conditioning.

Verifier design

Use independent measurements for task success where possible: calibrated object pose, support/relation checks, grasp evidence, fixture state, and recorded video reviewed under a predefined rubric. Express uncertainty and observation failures. Evaluate reward classifiers and automatic verifiers against manually adjudicated samples, including borderline failures.

Choose tolerance and dwell-time criteria before final runs. The current simulation guard's 15 mm threshold is an implementation setting for a specific transit check, not a universal benchmark success tolerance. Camera accuracy and task geometry determine valid physical thresholds.

Statistical reporting

- Pilot variance and failure frequency before selecting final sample sizes.
- Report confidence intervals, per-task/per-robot results, and aggregated results with their weighting rule.
- Account for repeated trials within robot, day, object, or task; use clustered analysis or an appropriate hierarchical model rather than pretending every attempt is independent.
- For budget-limited recovery, use a prespecified censored-time analysis or report fixed-budget success and restricted mean attempts; do not discard unrecovered episodes.
- Predefine the primary comparison and distinguish exploratory secondary analyses.

For perspective, **20 successes in 20 independent, identically distributed trials gives a one-sided exact 95% lower bound of approximately 86%: $0.05^{(1/20)} \approx 0.861$** . It does not establish 99% reliability, and dependence between repeated trials further limits the interpretation.

The existing BotCortex computation/playback timings demonstrate why latency categories matter. A simulator running faster or slower than real time should not silently change the reported robot execution speed.

12. Implementation roadmap and acceptance gates

The schedule below is an engineering estimate, assuming reliable access to two arms, necessary sensors, and sufficient support. It is not a guarantee for one person working around procurement or hardware downtime.

Phase	Estimate	Work and acceptance gate
0. Scope and hardware	1–2 weeks	Bring up the RoArm-M2-Pro controller link; mount and calibrate the RealSense (camera-to-base transform for each arm, timestamp source, measured pose accuracy); freeze initial tasks, fault taxonomy, evidence schema, and pilot protocol.
1. Adapters and perception	3–4 weeks	Two real adapters pass conformance; frames/timestamps validated; bounded motion and stopping tested; outcome sensing works.
2. Contracts and benchmark	3 weeks	Versioned contracts, replay packets, controlled faults, independent verifier, and simple baselines run end-to-end.

Phase	Estimate	Work and acceptance gate
3. Diagnosis and selection	3–4 weeks	Passive and active selectors share repair operators; ambiguity/abstention supported; simulation pilot shows measurable value.
4. Hardware evaluation	4–6 weeks	Freeze splits/configuration; run core comparisons and justified ablations; include unsuccessful repairs and natural failures.
5. Paper and artifacts	2 weeks	Statistical analysis, figures, claim audit, reproducibility package, videos, and submission formatting.

The phases sum to approximately **16–21 weeks** if largely sequential. Some analysis and writing can overlap; procurement, debugging, and experimental resets can extend the calendar.

Concrete engineering work packages

1. **Capability schema:** remove global platform assumptions and define versioned units, frames, modes, limits, sensors, and gripper semantics. Gate: instantiate both platforms without shared-state contamination.
2. **Adapter conformance:** test command/state mapping, timestamps, bounded control, interruption, and unavailable capabilities. Gate: the same suite passes both adapters with explicit capability-specific expectations.
3. **Perception bridge:** replace privileged simulation lookups with a shared observation contract. Gate: identical task code can consume simulated and measured estimates, with uncertainty preserved.
4. **Contract engine:** support preconditions, effects, invariants, and unknown verdicts. Gate: intentional pushing is allowed while bystander damage is rejected.
5. **Evidence recorder:** persist synchronized actual actions, observations, versions, and overrides. Gate: a disputed primitive can be reconstructed and reviewed without agent narration.
6. **Replay and fault harness:** snapshot state, inject bounded faults, and compare hypotheses. Gate: diagnostic queries are non-mutating and repeatable within stated numerical tolerances.
7. **Repair operators:** implement bounded, versioned program and calibration/physics repairs first. Gate: each operator has a measurable cost and independently verified outcome.
8. **Selector and active probes:** establish fixed/rule-based baselines before a learned selector. Gate: active probes add value after accounting for their cost.
9. **Memory isolation:** condition retrieval on embodiment/version and enforce split boundaries. Gate: held-out traces cannot leak through the retrieval index.
10. **Evaluation runner and paper artifact:** freeze configurations and export auditable episodes/statistics. Gate: rerunning analysis reproduces tables and includes failed/budget-exhausted cases.

Go/no-go decisions

- **After phase 1:** if the RealSense does not give reliable object-state verification at the task tolerances, or the RoArm-M2-Pro cannot execute the shared tasks, resolve that dependency before claiming a two-robot study.

- **After phase 2:** if controlled failure classes cannot be distinguished even with extra sensing, narrow the taxonomy or study uncertainty rather than forcing labels.
- **After phase 3:** if simple rules match the proposed selector, improve the research hypothesis before expanding physical trial count.
- **Before submission:** if only simulation is complete, present a simulation study with that scope; do not describe promised hardware experiments as results.

13. Resources, prerequisites, and main risks

Hardware and instrumentation

The second arm is the Waveshare RoArm-M2-Pro; the shared sensor is an Intel RealSense depth camera that is owned but not yet mounted. Until it is mounted and calibrated, the OpenArm rig remains camera-less and verification is proprioceptive only, so the perception and frame/TCP fault classes cannot be exercised on hardware. Neither arm has a force/torque sensor. Restrict contact-related claims and tasks accordingly; motor position error alone is not general contact-force measurement, and a contact-sensitive insertion task is admissible only if the RealSense plus proprioception can verify its outcome.

Bring-up requires calibrated RealSense-to-base and tool transforms for each arm, validated units and joint ordering (including the RoArm-M2-Pro's servo-tick to radian conversion), suitable grippers, repeatable objects/fixtures, synchronized recordings across the Thor host and the RoArm-M2-Pro controller, and independent stop facilities for both arms. Real motion remains supervised and explicitly authorized. No hardware connection or motion was needed for this dossier.

Compute and data

Use existing primitives and planners for early experiments, then add small task policies only where needed. Run bulk simulation headlessly. Budget separately for parameter search, policy adaptation, authoring calls, data storage, and human annotation. Keep pretrained-model and demonstration costs in comparisons even when amortized.

Play2Perfect's week-long pretraining and per-task finetuning illustrate a different compute commitment from this proposed selector study. It is a useful later executor, not a prerequisite for the first paper. [1](#)

Risks and mitigation

Risk	Practical response
Observationally equivalent causes	Add targeted measurements and calibrated abstention; avoid forced single labels.
Simulator explains the wrong mechanism	Use uncertainty ensembles and validate predicted intervention effects on hardware.

Risk	Practical response
Weak or circular verifier	Independently adjudicate representative successes, failures, and ambiguous cases.
Repair baseline is artificially weak	Share operators, observations, and budgets; include simple rules and strong adapted methods.
Cross-robot differences dominate	Match task semantics, report capabilities, and separate calibration from program changes.
Memory contaminates held-out evaluation	Freeze indexes/splits and audit artifact provenance.
Trial workload overwhelms schedule	Pilot first; prioritize primary comparisons; record resets and diagnostic attempts.
Apparent novelty disappears	Update the closest-work review and narrow the contribution before submission.

14. Publication strategy

Choose the venue based on demonstrated evidence. A strong learning/decision method with real transfer results can fit **CoRL** or **RSS**. A rigorous robotic method and integrated system can fit **ICRA** or **IROS**. A workshop is useful for early feedback, but is not equivalent to acceptance of a full conference paper.

Deadline snapshot — checked 11–12 September 2026

Venue	Verified planning information	Implication
ICRA 2027	Official CFP lists 15 September 2026 as the paper deadline. 23	Too soon for the proposed new hardware study.
CoRL 2026	Submission window has passed; the author instructions describe double-blind review and an initial 8-page paper. 24 , 25	Use for formatting expectations, not as an available submission window.
RSS 2027	Announced 6–11 July 2027 in Athens ; a 2027 paper deadline was not verified. 26	Track the official CFP; do not infer a date from a prior year.
IROS 2027	IEEE RAS event listing gives 1 March 2027 for paper submission. 27	Potential planning target; confirm against the final conference CFP.

Dates are a historical snapshot and must be rechecked before scheduling a submission. No CoRL 2027 deadline is asserted here.

Submission package

The paper should contain a precise problem definition, clear separation from Fail2Progress/COMPASS/ASID and program-repair work, a fully specified selector, strong equal-budget baselines, two-robot results, ablations, uncertainty analysis, and honest negative cases. Release enough configurations, code, traces, and analysis for another lab to understand what changed between attempts.

Keep the method contribution separate from product positioning. “Teach your robot by typing” explains the product; “fewer attempts to recover through evidence-based intervention selection” is a testable paper claim.

The recommended target is a credible **2027 submission**, with venue choice following the actual result rather than rushing to an imminent deadline. Acceptance cannot be predicted from a proposal.

15. Claim ledger and final recommendation

Claim	Current status	Evidence needed to strengthen it
The captured blue-block simulation failure was fixed.	Supported for the reproduced task and tested variants.	Broader task/perturbation benchmark for generalization.
Native and browser paths share useful runtime behavior.	Supported by the recorded regression checks.	Expanded semantic conformance and performance characterization.
BotCortex is already a hardware-portable SDK.	Not established.	Two implemented real adapters, sensing, binding, and conformance evidence.
The same task program can transfer across both selected arms.	Research objective.	Frozen task hashes plus separately reported adapter/calibration changes.
Active diagnosis reduces recovery effort.	Primary hypothesis.	Equal-budget physical comparisons and statistical analysis.
Replay can always identify the root cause.	Not a defensible claim.	Use bounded identifiability statements and unresolved cases.
Browser simulation or skill memory is new.	Contradicted by close prior work.	Novelty must come from the specific decision method and evidence.
A small perfect trial set proves near-perfect reliability.	Unsupported.	Appropriate uncertainty estimates and larger, diverse trials.
A major-conference paper is guaranteed.	Not knowable.	Strong completed research and peer review.

Recommended next implementation milestone

Establish **one objectively verified task on two physical arms**, with the same task intent and explicitly versioned adapters/calibration. Capture failures without conflating program, perception, calibration, and dynamics. Then compare a simple intervention selector against always-rewrite and always-calibrate under the same budget.

That milestone provides an honest foundation for both the SDK and the proposed paper. Dexterous pretraining, a large skill marketplace, and broad fleet transfer can build on it later without enlarging the first research question.

16. Sources and research notes

References are linked to primary papers, repositories, or official conference pages. “Core review” means the relevant main-text methods/results and selected supporting sections were examined during the research, not that every appendix or citation was independently reproduced. Background entries support context and further reading; they are not evidence for unreported BotCortex performance.

[1] Play2Perfect

Play2Perfect: What Matters in Dexterous Play Pretraining for Precise Assembly? arXiv 2606.26428, reviewed **v3 (8 September 2026)**. Core review, including training setup, hardware results, limitations, and selected appendices.

- Paper: <https://arxiv.org/abs/2606.26428v3>
- Full text: <https://arxiv.org/html/2606.26428v3>
- Project: <https://play2perfect.github.io/>
- MIT-licensed code: <https://github.com/kushal2000/play2perfect>
- Deployment guide:
<https://github.com/kushal2000/play2perfect/blob/main/docs/deployment.md>

[2] DexGPT

Hu-xiao-max/dexgpt, repository snapshot **03ba8a26eaf9dfd272ac1a22c0fd06ccb9b2e5b**. Repository/code/report review; no top-level license identified at the snapshot.

- Pinned repository: <https://github.com/Hu-xiao-max/dexgpt/tree/03ba8a26eaf9dfd272ac1a22c0fd06ccb9b2e5b>
- Inspect `dexgpt/motion/retarget.py`, `dexgpt/simulation/rollout.py`, and `outputs/physics_report.json` in that snapshot.

[3] ASPIRE

ASPIRE, arXiv **2607.00272**. Core review of program search, trace feedback, reusable skills, and transfer examples. A close comparison for program repair and skill-library claims.

<https://arxiv.org/abs/2607.00272>

[4] CaP-X

CaP-X, arXiv **2603.22435**, reviewed **v2**. Core review of coding-agent evaluation and the perception/control harness.

<https://arxiv.org/abs/2603.22435v2>

[5] Nautilus

Nautilus, arXiv **2605.11665**. Core interface/architecture review. Relevant to typed contracts, isolated execution, and the distinction between portable interfaces and successful policy transfer.

<https://arxiv.org/abs/2605.11665>

[6] Fail2Progress

Yixuan Huang, Novella Alvina, Mohanraj Devendran Shanthi, and Tucker Hermans. *Fail2Progress: Learning from Real-World Robot Failures with Stein Variational Inference*. arXiv **2509.01746v1**, CoRL 2025. Core failure-reconstruction and limitation review; bibliographic metadata rechecked for this dossier.

<https://arxiv.org/abs/2509.01746v1>

[7] Counterexample-guided abstraction repair

Counterexample-guided abstraction repair, arXiv **2105.06537**. Targeted closest-work review of counterexample-based symbolic/geometric repair.

<https://arxiv.org/abs/2105.06537>

[8] COMPASS / What Went Wrong?

Peide Huang and colleagues. *What Went Wrong? Closing the Sim-to-Real Gap via Differentiable Causal Discovery*. CoRL 2023, PMLR **229:734–760**. Targeted causal-calibration review; official proceedings metadata and abstract rechecked for this dossier.

- Proceedings: <https://proceedings.mlr.press/v229/huang23c.html>
- Paper: <https://proceedings.mlr.press/v229/huang23c/huang23c.pdf>

[9] RAPT

RAPT: Model-Predictive Out-of-Distribution Detection and Failure Diagnosis for Sim-to-Real Humanoid Deployment. arXiv **2602.01515**. Targeted review of prediction, localization, and diagnosis.

<https://arxiv.org/abs/2602.01515>

[10] ASID

ASID: Active Exploration for System Identification in Robotic Manipulation. arXiv **2404.12308**. Core review of active physical identification.

<https://arxiv.org/abs/2404.12308>

[11] Phys2Real

Phys2Real: Fusing VLM Priors with Interactive Online Adaptation for Uncertainty-Aware Sim-to-Real Manipulation. arXiv **2510.11689**, reviewed **v2**. Core review of physical priors and interactive adaptation.

<https://arxiv.org/abs/2510.11689v2>

[12] Neural Fidelity Calibration

Neural Fidelity Calibration for Informative Sim-to-Real Adaptation. arXiv **2504.08604**. Targeted review of informative simulator adaptation.

<https://arxiv.org/abs/2504.08604>

[13] TRANSIC

TRANSIC: Sim-to-Real Policy Transfer by Learning from Online Correction. arXiv **2405.10315**, reviewed **v3**. Core review of base-policy transfer, gated residuals, and human corrections.

<https://arxiv.org/abs/2405.10315v3>

[14] HIL-SERL

HIL-SERL, arXiv **2410.21845**. Core paper/project review of direct hardware learning, demonstrations, interventions, and reward classifiers.

- Paper record: <https://arxiv.org/abs/2410.21845>
- Project: <https://hil-serl.github.io/>
- Project paper: <https://hil-serl.github.io/static/hil-serl-paper.pdf>

[15] RialTo

RialTo: A Real-to-Sim-to-Real Approach for Robust Manipulation. arXiv **2403.03949**. Abstract/project review for reconstructed-environment robustification; no experiment reproduced here.

- Paper: <https://arxiv.org/abs/2403.03949>
- Project: <https://real-to-sim-to-real.github.io/RialTo/>

[16] CEI

CEI: A Unified Interface for Cross-Embodiment Visuomotor Policy Learning in 3D Space. arXiv **2601.09163**. Core representation/trajectory-alignment review, including functional representations and directional Chamfer distance.

<https://arxiv.org/abs/2601.09163>

[17] UHAS

Cross-Embodiment Robot Manipulation via a Unified Hand Action Space. arXiv **2607.03570**. Core transfer/limitations review. The reviewed real LEAP setup still required identified gains/velocity limits and matched **20 Hz** control because of serial bandwidth; shared action space did not remove hardware calibration.

<https://arxiv.org/abs/2607.03570>

[18] ReKep

ReKep: Spatio-Temporal Reasoning of Relational Keypoint Constraints for Robotic Manipulation. arXiv **2409.01652**. Targeted review of relational constraints and closed-loop optimization.

<https://arxiv.org/abs/2409.01652>

[19] Grounding Sim-to-Real Generalization

Grounding Sim-to-Real Generalization, arXiv **2603.22876**. Supporting transfer-evaluation source; full text was collected, with a narrower review than the closest-work papers.

<https://arxiv.org/abs/2603.22876>

[20] DrEureka

DrEureka, arXiv **2406.01967**. Background discovery/abstract-project review for language-assisted reward and domain-randomization design.

<https://arxiv.org/abs/2406.01967>

[21] Re³Sim

Re³Sim, arXiv **2502.08645**. Background discovery/abstract review for real-to-sim reconstruction and manipulation data generation.

<https://arxiv.org/abs/2502.08645>

[22] RobolInspector

RobolInspector, arXiv **2508.21378**. Supporting source for failure-diagnostic feedback; also cited in the existing company/runtime research. Its reported gains are not reproduced in this dossier.

<https://arxiv.org/abs/2508.21378>

[23] ICRA 2027 official call for papers

Official submission deadline source, checked during the 11–12 September 2026 research snapshot.

<https://2027.ieee-icra.org/contribute/call-for-icra-2027-papers-now-accepting-submissions/>

[24] CoRL author instructions

Official author guidance consulted for review format and initial paper length; future-year rules may differ.

<https://www.corl.org/contributions/instruction-for-authors>

[25] CoRL call for papers

Official conference contribution information for the 2026 window. Recheck the site for the next cycle.

<https://www.corl.org/contributions/call-for-papers>

[26] Robotics: Science and Systems

Official conference site announcing RSS 2027 in Athens, 6–11 July. A 2027 paper deadline was not verified in this review.

<https://roboticsconference.org/>

[27] IROS 2027 IEEE RAS event listing

Official society event listing consulted for the 1 March 2027 paper-submission date; confirm against the final conference CFP.

<https://www.ieee-ras.org/event/2027-ieee-rsj-international-conference-on-intelligent-robots-and-systems-iros-70525/>

Local evidence and reproducibility notes

- `docs/SIMULATION_FAILURE_FIX.md` : captured failure, native/WASM outcomes, timings, verification history, source commit, and artifact provenance.
- `docs/ARCHITECTURE_AUDIT.md` and `docs/CLAUDE_REVIEW.md` : earlier architecture/lifecycle review and fixes; historical context rather than hardware research results.

- `public/botcortex/MANIFEST.json`: shipped runtime/wheel pin. Recorded wheel SHA-256: `c657c6e094e7db4970068b7ff2ee3eb71c189335855961a474bf084054e1387d`.
- Sibling `botcortex-runtime` source: `platform.py`, `config.py`, `jointmap.py`, `robot.py`, `kinematics.py`, `scene.py`, `skills.py`, `session.py`, `sim.py`, and `wasm.py` establish the inspected implementation boundaries.
- `.firecrawl/sim2real/`: ignored working research cache, including anchor full texts and thematic search exports. The cited public sources above remain the portable reference record.

End of dossier. Research and deadline statements reflect the 12 September 2026 snapshot. Proposed architecture, timelines, and experiments remain subject to the stated implementation and evidence gates.